Contents

Networking — Deep Dive Notes **1**
1. OSI Model (7 Layers) . . . 1
2. TCP Deep Dive . . . 2
   - Three-Way Handshake . . . 2
   - Connection Termination (Four-way) . . . 2
   - TCP States (Important for Troubleshooting) . . . 2
   - TCP vs UDP . . . 2
3. IP Addressing & Subnetting . . . 2
   - CIDR Notation . . . 2
   - Private Ranges (RFC 1918) . . . 2
   - Subnetting Quick Math . . . 3
4. DNS . . . 3
   - Record Types . . . 3
   - DNS Resolution Flow . . . 3
   - Troubleshooting DNS . . . 3
5. Load Balancers . . . 3
   - Layer 4 (Transport Layer) LB . . . 3
   - Layer 7 (Application Layer) LB . . . 3
   - Load Balancing Algorithms . . . 4
   - Health Checks . . . 4
6. NGINX as Reverse Proxy / Load Balancer . . . 4
7. TLS/SSL & Certificate Management . . . 5
   - TLS Handshake (TLS 1.3) . . . 5
   - Certificate Types . . . 5
   - Practical TLS . . . 5
8. Firewalls & Security Groups . . . 5
   - AWS Security Groups . . . 5
   - iptables Chains . . . 5
9. VPN & Network Tunneling . . . 5
   - VPN Types . . . 5
   - WireGuard Quick Setup . . . 6
10. Service Mesh Networking (Istio/Linkerd) . . . 6
    - Concepts . . . 6
    - What a Service Mesh Provides . . . 6
- Resources . . . 6

Networking — Deep Dive Notes

1. OSI Model (7 Layers)

Layer	Name	Protocol/Example	PDU
7	Application	HTTP, DNS, SMTP, SSH	Data
6	Presentation	TLS/SSL, JPEG encoding	Data
5	Session	RPC, NetBIOS	Data
4	Transport	TCP, UDP, SCTP	Segment
3	Network	IP, ICMP, OSPF	Packet
2	Data Link	Ethernet, ARP, MAC	Frame
1	Physical	Cables, Wi-Fi signals	Bits

DevOps Focus Areas: Layer 3 (routing), Layer 4 (ports, load balancing), Layer 7 (HTTP proxying)

---

2. TCP Deep Dive

Three-Way Handshake

```
Client → Server: SYN (seq=x)
Server → Client: SYN-ACK (seq=y, ack=x+1)
Client → Server: ACK (seq=x+1, ack=y+1)
```

Connection Termination (Four-way)

```
Client → Server: FIN
Server → Client: ACK
Server → Client: FIN
Client → Server: ACK (→ TIME_WAIT 2*MSL=60s)
```

TCP States (Important for Troubleshooting)

State	Meaning
LISTEN	Waiting for connections
ESTABLISHED	Active connection
TIME_WAIT	Waiting after close (60s) — can exhaust ports
CLOSE_WAIT	Remote end closed, local hasn't yet
FIN_WAIT_2	Sent FIN, waiting for remote FIN

```
ss -tn state established
ss -tn state time-wait | wc -l    # high = port exhaustion risk

# Tune TIME_WAIT
echo 1 > /proc/sys/net/ipv4/tcp_tw_reuse
```

TCP vs UDP

Feature	TCP	UDP
Reliable	Yes	No
Ordered	Yes	No
Connection	Yes (handshake)	No (connectionless)
Flow control	Yes	No
Use cases	HTTP, SSH, DB	DNS, video streaming, games

---

3. IP Addressing & Subnetting

CIDR Notation

```
10.0.0.0/24  → 256 addresses (254 usable), mask: 255.255.255.0
10.0.0.0/22  → 1024 addresses, mask: 255.255.252.0
10.0.0.0/16  → 65536 addresses
192.168.1.0/30 → 4 addresses (2 usable) — point-to-point links
```

Private Ranges (RFC 1918)

```
10.0.0.0/8        → Class A private
172.16.0.0/12     → Class B private
192.168.0.0/16    → Class C private
169.254.0.0/16    → Link-local (APIPA)
127.0.0.0/8       → Loopback
```

Subnetting Quick Math

```
/24 = 256 hosts    /25 = 128    /26 = 64    /27 = 32    /28 = 16
Formula: 2^(32-prefix) = total addresses; subtract 2 for network + broadcast
```

---

4. DNS

Record Types

Type	Purpose	Example
A	IPv4 address	example.com → 93.184.216.34
AAAA	IPv6 address	example.com → 2606:...
CNAME	Alias to another name	www → example.com
MX	Mail server	example.com → mail.example.com (priority)
TXT	Arbitrary text (SPF, DKIM, ACME challenge)	
NS	Nameserver	example.com → ns1.example.com
PTR	Reverse DNS (IP → name)	
SRV	Service discovery	_http._tcp.example.com
SOA	Start of authority	Zone info, TTL, serial

DNS Resolution Flow

```
Browser → OS resolver cache → /etc/hosts → /etc/resolv.conf nameserver
→ Recursive resolver (e.g., 8.8.8.8) → Root NS → TLD NS → Auth NS → Answer
```

Troubleshooting DNS

```
dig example.com A +short
dig @8.8.8.8 example.com ANY         # query specific server
dig +trace example.com                # trace full resolution
nslookup example.com
host example.com
dig -x 93.184.216.34                  # reverse lookup

# Check local resolution
cat /etc/resolv.conf
cat /etc/nsswitch.conf | grep hosts   # order: files dns (or dns files)
```

---

5. Load Balancers

Layer 4 (Transport Layer) LB

- Routes based on IP + port
- No content inspection
- Faster, less CPU overhead
- Examples: AWS NLB, HAProxy (TCP mode)

Layer 7 (Application Layer) LB

- Routes based on HTTP headers, URL paths, cookies
- SSL termination, content rewriting
- Examples: AWS ALB, NGINX, HAProxy (HTTP mode), Envoy

Load Balancing Algorithms

Algorithm	Description	Use Case
Round Robin	Rotate through servers	Simple, equal servers
Least Connections	Send to server with fewest connections	Variable request duration
IP Hash	Hash client IP → consistent server	Session affinity without cookies
Random	Random selection	Simple, low overhead
Weighted Round Robin	Some servers handle more traffic	Heterogeneous servers
Resource Based	Route by CPU/memory availability	Dynamic environments

Health Checks

```
Active: LB polls backend (TCP connect, HTTP GET /health)
Passive: LB monitors real traffic for failures
```

---

6. NGINX as Reverse Proxy / Load Balancer

```
# /etc/nginx/conf.d/app.conf
upstream backend {
    least_conn;
    server app1:8080 weight=3;
    server app2:8080 weight=1;
    server app3:8080 backup;    # only used if others fail
    keepalive 32;               # keep connections warm
}

server {
    listen 443 ssl http2;
    server_name api.example.com;

    ssl_certificate /etc/ssl/certs/cert.pem;
    ssl_certificate_key /etc/ssl/private/key.pem;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 5s;
        proxy_read_timeout 60s;
        proxy_buffering off;
    }

    location /static/ {
        root /var/www;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}
```

---

7. TLS/SSL & Certificate Management

TLS Handshake (TLS 1.3)

```
Client → Server: ClientHello (supported ciphers, TLS version)
Server → Client: ServerHello + Certificate + CertificateVerify
Client verifies cert against CA trust store
Key exchange (ECDHE) → session keys
Both parties: Finished
```

Certificate Types

- **DV (Domain Validated):** Automated, proves domain ownership
- **OV (Org Validated):** Proves organization identity
- **EV (Extended Validation):** Highest trust, shows company name

Practical TLS

```bash
# Check certificate details
openssl s_client -connect api.example.com:443 -servername api.example.com
openssl x509 -text -noout -in cert.pem
openssl x509 -enddate -noout -in cert.pem    # expiry

# Generate self-signed cert
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem \
  -days 365 -nodes -subj '/CN=localhost'

# Let's Encrypt (Certbot)
certbot certonly --webroot -w /var/www/html -d example.com -d www.example.com
certbot renew --dry-run

# mTLS — mutual authentication
# Both client and server present certificates
# Used in service meshes, internal microservices
```

---

8. Firewalls & Security Groups

AWS Security Groups

```
Stateful: If inbound rule allows traffic, response is automatically allowed
Direction: Inbound and outbound separate
Default: Deny all inbound, allow all outbound
Groups: Can reference other security groups
```

iptables Chains

```
INPUT → Traffic to this host
OUTPUT → Traffic from this host
FORWARD → Traffic passing through (router mode)
PREROUTING → Before routing decision (NAT DNAT)
POSTROUTING → After routing decision (NAT SNAT/MASQUERADE)
```

---

9. VPN & Network Tunneling

VPN Types

- **Site-to-Site:** Connect two networks (e.g., office to cloud VPC)

- **Client-to-Site:** Individual user connects to network (OpenVPN, WireGuard)

WireGuard Quick Setup

```
# Generate keys
wg genkey | tee privatekey | wg pubkey > publickey

# /etc/wireguard/wg0.conf
[Interface]
Address = 10.100.0.1/24
ListenPort = 51820
PrivateKey = <server_private_key>

[Peer]
PublicKey = <client_public_key>
AllowedIPs = 10.100.0.2/32

wg-quick up wg0
```

---

10. Service Mesh Networking (Istio/Linkerd)

Concepts

- **Sidecar proxy:** Envoy runs alongside each pod, intercepts all traffic
- **Control plane:** Manages configuration for all proxies (Istiod)
- **Data plane:** All sidecar proxies handling actual traffic

What a Service Mesh Provides

- mTLS between all services automatically
- Traffic management: retries, timeouts, circuit breakers
- Observability: metrics, tracing, logging per-service
- Traffic splitting: canary deployments without code changes

```
# Istio VirtualService — weighted routing
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
      weight: 90
    - destination:
        host: reviews
        subset: v2
      weight: 10
```

Resources

- https://beej.us/guide/bgnet/ — Beej's Network Guide (free)
- https://www.cloudflare.com/learning/ — Excellent networking articles
- https://jvns.ca/networking-zine.pdf — Julia Evans networking zine

- https://www.nginx.com/resources/wiki/ — NGINX config reference
- https://istio.io/latest/docs/ — Istio documentation